



LISTA DE EXERCÍCIOS 4

GABARITO...

OBS. Resoluções em sala de aula podem englobar mais pontos de vista e atributos do que apenas as respostas simplificadas apresentadas no gabarito.

1. Forneça três exemplos no qual *multithreading* oferece melhor desempenho do que uma solução baseada em thread única.

Resposta: (1) Um servidor web que responde cada requisição em uma thread separada; (2) Uma aplicação paralelizada como por exemplo, para a multiplicação de matrizes, onde diferentes padrões de matrizes podem ser trabalhados em paralelo; (3) Um programa de GUI interativa como no caso de um debugger, onde uma thread é usada para monitorar a entrada do usuário, outra thread é usada para representar a aplicação em execução, e uma terceira para monitorar o desempenho.

2. Quais são as diferenças entre threads de nível de usuário e threads de nível de kernel? Sob quais circunstâncias um tipo é melhor que o outro?

Resposta: Threads de nível de usuário são desconhecidas pelo kernel, enquanto as de nível de kernel, são conhecidas. Em sistemas com mapeamento M:1 ou M:N, threads de usuário são escalonadas pela biblioteca de thread e o kernel escala threads de kernel. Threads de kernel não precisam estar associadas com um processo, enquanto cada thread de usuário pertence a um processo. Threads de kernel são geralmente mais “caras” de manter do que threads de usuário, considerando que estas devem ser representadas com uma estrutura de dados em nível de kernel.

3. Descreva as ações tomadas pelo kernel para realizar o chaveamento de contexto entre threads em nível de kernel.

Resposta: O chaveamento de contexto entre threads de kernel tipicamente requer salvar o valor dos registradores de CPU da thread sendo interrompida (switched out) e restaurar o conteúdo dos registradores da CPU a partir do conteúdo da nova thread sendo escalonada.

4. Quais recursos são usados quando uma *thread* é criada? Como eles são diferentes daqueles usados quando um processo é criado?

Resposta: Considerando que uma thread é menor que um processo, a criação de thread usa tipicamente menos recursos do que a criação de um processo. Dessa forma, parte-se do arcabouço criado no momento da criação do processo, o qual inclui um bloco de controle de processo (Process Control Block ou PCB), que é uma estrutura de dados relativamente grande. O PCB inclui um mapa de memória, a lista de arquivos abertos e variáveis de ambiente. Alocar e gerenciar o mapa de memória é tipicamente a atividade mais custosa em relação a consumo de tempo. Considerando a atividade de criação de thread (partindo do pressuposto que o processo já existe), criar uma thread de espaço de usuário ou de espaço de kernel envolve a alocação de uma estrutura de dados menor para manter um conjunto de

registradores (register set), pilha (stack) e prioridade específicos de cada thread.

5. Assuma que que um sistema operacional mapeia threads de nível de usuário para o kernel usando o modelo muitos-para-muitos e que o mapeamento seja feito por meio de LWPs. Além disso, o sistema permite que desenvolvedores criem threads de tempo real para uso em sistemas de tempo real. É necessário atrelar (*bind*) uma thread de tempo real a um LWP? Explique.

Resposta: Sim. Tempo é crucial em aplicações de tempo real. Se um thread é marcado como de tempo real, mas não é atrelado a um LWP, a thread pode ter que esperar para ser atrelada a um LWP antes de ser executada. Considere que uma thread de tempo real está em execução (é atrelada a um LWP) e então é bloqueada (i.e. deve realizar I/O, foi interrompida por uma thread de tempo real de maior prioridade, está esperando por um lock de exclusão mútua, etc). Enquanto a thread de tempo real está bloqueada, o LWP para o qual foi atrelada foi atribuído a outra thread. Quando a thread de tempo real foi escalonada para executar novamente, ela deve primeiro esperar para ser atrelada a um LWP. Pela ligação de um LWP a uma thread de tempo real, está se assegurando que a thread estará apta a executar com retardo (delay) mínimo uma vez que é escalonada.